

Personal Finance Tracker

Introduction to Programming 2 - Final Project

Presented by:

Kamila Ahmadzada

Samonov Iaroslav

Dinmukhamed Sailau

Abil Elaman

Problem Statement

Problem People struggle to track where their money goes and often overspend without realizing it.

Our Solution

A command-line Personal Finance Tracker that:

- Loads real transaction data from a JSON file*
- Categorizes all expenses automatically*
- Shows a monthly income/expense summary*
- Detects which categories went over budget*
- Lets users customize their own budget limits*



Program Menu

- 0. Set a budget*
- 1. Show all transactions*
- 2. Show categories*
- 3. Show monthly summary*
- 4. Show overspendings*
- 5. Exit*



Example Output(option 4)

shopping: overspent by \$125.0
food: overspent by \$95.0
entertainment: overspent by \$70.0
health: overspent by \$35.0

Program in Action

Key Technical Approaches

Dictionary for expense tracking

- $O(1)$ lookup speed instead of $O(n^2)$ nested loops
- category name is the key, total amount is the value

Generator in categories_expenses

- `only_expenses = (tr for tr in transactions if ...)`
- memory efficient — does not load all data at once

Abstract class for filter

- `Filter` is a base class, `FilterCategory` extends it
- demonstrates inheritance and polymorphism

Key Technical Approaches

Decorator in categories_expenses

- *wraps the load function with error handling*
- *advanced Python feature — reusable error wrapper*

Encapsulation in Transaction class

- *private attributes with getters (getType, getAmount...)*
- *protects data from being changed accidentally*

Testing

Test 1: Overspending detected correctly

→ food budget \$50, spent \$80

→ over_by should be \$30

Test 2: Empty transactions

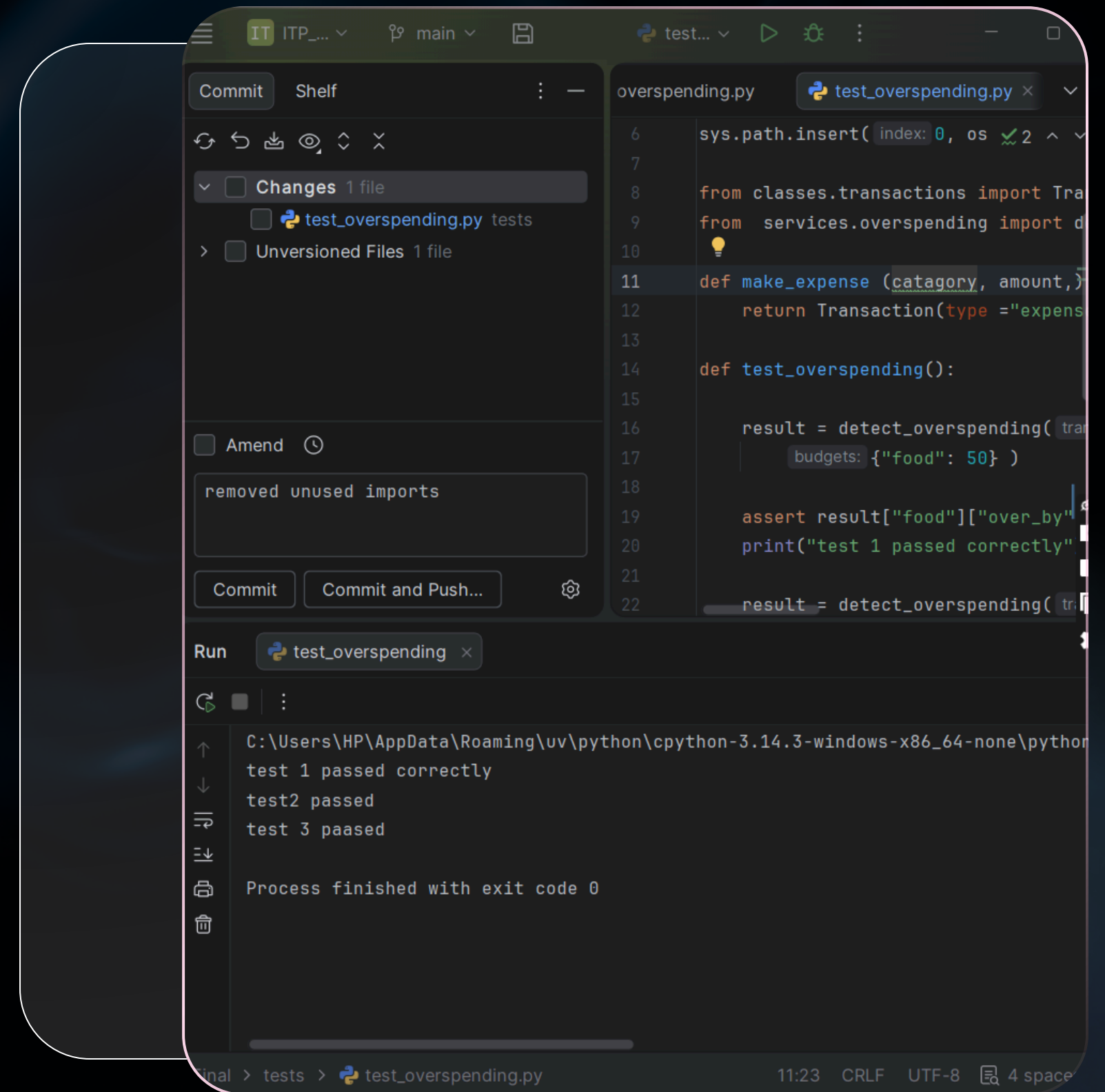
→ no transactions

→ result should be empty {}

Test 3: Exactly at the limit

→ spent \$50, budget \$50

→ NOT overspending



The screenshot shows a code editor with two files: `overspending.py` and `test_overspending.py`. The `test_overspending.py` file contains the following code:

```
6 sys.path.insert(index=0, os=2 ^
7
8 from classes.transactions import Tra
9 from services.overspending import d
10
11 def make_expense (catagory, amount,)
12     return Transaction(type ="expens
13
14 def test_overspending():
15
16     result = detect_overspending( tra
17         budgets: {"food": 50} )
18
19     assert result["food"]["over_by"
20     print("test 1 passed correctly"
21
22     result = detect_overspending( tr
```

The `Run` panel at the bottom shows the output of the tests:

```
test 1 passed correctly
test2 passed
test 3 paased
Process finished with exit code 0
```

Team Contribution

Kamila Ahmadzada

- services/overspending.py — detect_overspending, print_overspending_report
- tests/test_overspending.py — 3 unit tests with edge cases

Samonov Iaroslav

- distribution of work, work with the directorate
- Parser
- UI
- Filter → FilterData

Sailau Dinmukhammed

- incremented encapsulation
- Filter class → FilterCategory
- monthlySummary

Abil Elaman

- Generator
- Expences_categories
- __repe__